PROJECT REPORT

DATA CLEANING USING EXCEL AND PYTHON

Identification and Removal of Missing Values and Duplicate Records

Submitted by

Mohamed Isrin A

Department of Information Technology

Academic Project Report

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope
6. Tools and Technologies
7. System Requirements
8. Dataset Description
9. Data Quality Issues
10.Workflow Design
11. Importing the Dataset
12. Initial Profiling
13. Handling Missing Values
14.Removing Duplicates
15.Standardizing Text
16. Validating Numeric Values
17.Documenting Changes
18. Exporting to Excel
19. Post-Cleaning Validation
20. Results and Observations
21. Before-and-After Comparison
22. Benefits of Automation
23. Limitations
24. Security and Ethics
25. Testing Plan
26. Implementation Steps
27.Future Enhancements
28. Conclusion and References

Abstract

This project demonstrates a practical method for improving the quality of a small dataset using Python and Excel. Raw spreadsheets may contain missing values, duplicate rows, inconsistent capitalization and unwanted spaces. These problems can affect later calculations and reporting. The project uses a fictional customer dataset, inspects its quality, applies documented cleaning rules and exports a cleaned Excel workbook. A change log records the work performed. The aim is to show a repeatable beginner-level workflow that can be adapted to student, inventory, customer or survey data.

Introduction

Data cleaning is the process of identifying and handling errors, gaps and inconsistencies in a dataset before analysis. A spreadsheet can appear correct while still containing hidden spaces, repeated records or values stored in the wrong format. These issues may lead to incorrect totals, misleading groupings and wasted time. Python provides a repeatable way to check and transform data, while Excel provides a familiar format for review and sharing. This report explains the stages from importing the raw file to validating and exporting the final result.

Problem Statement

Manually maintained spreadsheets can develop quality problems as records are copied, combined or entered by different people. Missing values can make calculations incomplete, duplicates can inflate counts, and inconsistent labels can split one category into several groups. Manual correction is difficult to repeat consistently. This project addresses the need for a simple, low-cost workflow that detects common problems, applies clear rules and documents each change. The original data is retained so that the cleaning process remains traceable.

Objectives

The primary objective is to prepare a consistent dataset and save it as an Excel workbook. The supporting objectives are to inspect rows and columns, measure missing values, detect exact duplicates, standardize text fields, validate numeric columns, export the result and document the changes. The project also aims to explain why each rule is used. Cleaning is not simply deleting every imperfect row; each decision should be appropriate for the intended use of the data.

Scope

The project focuses on basic cleaning of a small tabular dataset. It includes data profiling, missing-value handling, exact duplicate removal, text standardization, numeric validation and Excel export. It does not include advanced fuzzy matching, machine learning imputation, database pipelines or very large distributed datasets. A fictional customer dataset is used for learning. When adapting the method to real data, the user should confirm business rules and protect confidential information.

Tools and Technologies

Python is the programming language used for automation. Pandas provides DataFrame operations for reading, checking, transforming and exporting tabular data. Openpyxl supports Excel workbook creation through pandas. Excel or LibreOffice Calc can be used to inspect the final workbook. The code can run in Jupyter Notebook, Visual Studio Code or Google Colab. These tools are suitable for beginners and have extensive official documentation.

System Requirements

A normal laptop or desktop is sufficient because the sample dataset is small. The suggested software includes Python 3, pandas and openpyxl. A spreadsheet application is useful for checking the exported file but is not required to execute the Python cleaning steps. Before running the script, confirm that the source file is in the correct folder and that the column names match those used in the code. Google Colab can be used if Python is not installed locally.

Dataset Description

The demonstration dataset contains Customer_ID, Name, Age, City, Email and Purchase_Amount. Each row represents a fictional customer record. The sample is intentionally designed to include a repeated record, missing values and inconsistent text formatting. These examples allow the cleaning steps to be observed clearly. The dataset is not real customer information. The raw file should be preserved separately, and all cleaning should be performed on a working copy.

Customer_ID	Name	Age	City	Purchase_Amount
C001	Asha	22	Chennai	1200
C002	Ravi	Missing	chennai	850
C003	Meena	24	Madurai	Missing
C004	Arun	28	Trichy	1500
C004	Arun	28	Trichy	1500
C006	Sara	21	Missing	700

Data Quality Issues

Data quality includes completeness, uniqueness, consistency, validity and accuracy. Completeness asks whether required fields are present. Uniqueness checks for repeated records. Consistency checks whether the same concept is written in the same way. Validity checks whether values follow expected rules. The sample may contain a blank age, a blank purchase amount, city names with different capitalization and an exact duplicate row. These problems are recorded before making changes.

Workflow Design

The workflow starts by importing the source file and checking its dimensions, column names and data types. Next, missing values and duplicates are counted. A working copy is created so the original remains untouched. Cleaning rules are then applied: selected missing values are handled, exact duplicates are removed, text is standardized and numeric columns are validated. Finally, the output is rechecked and saved to Excel with a change log. This order makes the process easy to repeat.

Importing the Dataset

Pandas can read a CSV file with read_csv or an Excel workbook with read_excel. After loading, head() displays the first records, shape shows the number of rows and columns, and dtypes lists data types. These checks confirm that the expected file has been loaded. The filename in the code is an example and should be changed to match the actual file. The original file should not be overwritten during the initial import.

```
import pandas as pd

df = pd.read_csv('customer_data_raw.csv')
print('Shape:', df.shape)
print(df.head())
print(df.dtypes)
```

Initial Profiling

Profiling creates a baseline for comparison. The isnull().sum() command counts missing values in each column, while duplicated().sum() counts exact duplicate rows. The describe() method summarizes numeric columns. These results help determine which cleaning rules are needed. Recording the initial counts is important because it allows the final report to show what improved and what remains unresolved.

```
print(df.isnull().sum())
print('Duplicate rows:', df.duplicated().sum())
print(df.describe())
```

Handling Missing Values

A missing value should not automatically be replaced with zero or an average. The correct approach depends on what the field means and how the data will be used. For example, a missing purchase amount may mean the transaction was not recorded, while a missing city may be optional information. In this project, rows without a purchase amount can be excluded from purchase-value analysis, while other blanks may be retained when appropriate. The chosen rule must be documented.

```
clean = df.copy()
# Example policy: retain records needed for analysis
clean = clean.dropna(subset=['Purchase_Amount'])
print(clean.isnull().sum())
```

Removing Duplicates

Exact duplicate rows can cause records to be counted twice. Pandas provides duplicated() to identify repeated rows and drop_duplicates() to remove extra copies. This project removes exact duplicates only. Two customers with the same name are not necessarily the same person, so names alone should not be used as a duplicate key. The number of rows before and after removal should be recorded in the change log.

```
before = len(clean)
clean = clean.drop_duplicates()
print('Duplicates removed:', before - len(clean))
```

Standardizing Text

Text values may contain extra spaces or inconsistent capitalization. These differences can cause a city to appear as several separate categories during grouping. The strip() method removes surrounding spaces, while title() can standardize city labels and lower() can normalize email addresses. These transformations should be applied carefully because names and special spellings may require individual treatment. The raw source is retained for comparison.

```
for col in ['Name', 'City', 'Email']:
    if col in clean.columns:
        clean[col] = clean[col].astype('string').str.strip()
if 'City' in clean.columns:
    clean['City'] = clean['City'].str.title()
if 'Email' in clean.columns:
    clean['Email'] = clean['Email'].str.lower()
```

Validating Numeric Values

Numeric columns should be converted to numeric types and checked for values outside expected ranges. Age should be plausible, and purchase amounts should be non-negative for this demonstration. The rules depend on the dataset; negative values might represent returns in a sales system. Unusual values should be investigated instead of silently replaced. Conversion errors can be marked as missing and reviewed before deciding whether a row can be used.

Documenting Changes

A change log explains what was changed, why it was changed and how it was verified. It can record original rows, final rows, duplicates removed, missing values found and columns standardized. Actual counts should be taken from the script output rather than guessed. Documentation is valuable because cleaning decisions involve judgement. It allows a reviewer to understand the process and repeat it on a later file.

Exporting to Excel

Pandas can save the cleaned DataFrame to an .xlsx workbook using ExcelWriter. A useful workbook can contain one sheet for cleaned data and another for the change log. The index is generally omitted from the data sheet because it is not a source column. After export, open the workbook and check the headers, row count and values. A successful save message alone does not guarantee that the result is correct.

```
change_log = pd.DataFrame({
 'Item': ['Original rows', 'Final rows'],
 'Value': [len(df), len(clean)]
})
with pd.ExcelWriter('customer_data_cleaned.xlsx', engine='openpyxl') as w:
    clean.to_excel(w, sheet_name='Cleaned_Data', index=False)
    change_log.to_excel(w, sheet_name='Change_Log', index=False)
```

Post-Cleaning Validation

Validation checks whether the cleaning rules were followed. The final dataset should be checked for exact duplicates, remaining missing values and numeric data types. Any blanks that remain should be explained rather than hidden. The cleaned workbook should also be inspected in a spreadsheet application. If a validation check fails, the cause should be corrected and the script rerun from the beginning.

```
print('Final shape:', clean.shape)
print('Duplicates:', clean.duplicated().sum())
print(clean.isnull().sum())
print(clean.dtypes)
```

Results and Observations

The cleaning process should produce a more consistent dataset by removing repeated rows, trimming spaces and standardizing selected labels. The missing-value policy makes the final data appropriate for the intended analysis. The most important observation is that fewer rows do not automatically mean better data. Removing a row is justified only when a clear rule supports it. The actual counts depend on the input file and should be inserted after running the code.

Before-and-After Comparison

A before-and-after comparison can show the original row count, duplicate count, missing values and final row count. The city field should contain consistent labels after standardization, and exact duplicates should be reduced to zero if the removal step is successful. The report should use actual program output. A comparison helps demonstrate the effect of the workflow and highlights any remaining limitations.

Benefits of Automation

Automated cleaning improves consistency and reduces repetitive manual work. Once tested, the same script can be reused with future files by updating the file path and, if needed, the column names. The code can apply rules in the same order and create a repeatable change log. However, automation cannot determine the meaning of every missing or unusual value; human judgement and documented business rules are still necessary.

Limitations

The dataset is small and illustrative, and the rules may not apply to every real dataset. The script removes exact duplicates but does not identify similar records with spelling variations. Missing-value handling is also based on a simple example policy. Larger projects may require date parsing, domain-specific validation, duplicate matching and approval from a data owner. The report should not claim that the dataset is completely accurate merely because the code ran.

Security and Ethics

Data cleaning should protect privacy and preserve data integrity. Real customer information should be stored in approved locations and shared only with authorized people. This project uses fictional records to avoid exposing personal information. A value should not be changed merely to make a report appear better. Meaningful changes should be explainable, and the original file should be retained according to relevant data-retention rules.

Testing Plan

Testing confirms that the workflow performs as expected. The file-load test checks that required columns exist. The duplicate test verifies that exact duplicates are removed. The text test checks that spaces are trimmed, and the numeric test checks data types and ranges. The export test confirms that the workbook contains the expected sheets and opens correctly. A small dataset is useful because the expected result can be checked manually.

Test	Expected result	Verification
Load file	Expected columns present	Check shape and columns
Duplicates	Exact duplicates removed	duplicated().sum()
Text cleanup	Spaces trimmed	Inspect text columns
Numeric fields	Numeric types	Check dtypes
Export	Workbook opens	Open in Excel

Implementation Steps

Create a project folder and save the raw dataset as customer_data_raw.csv. Open a notebook or Python editor and install pandas and openpyxl if needed. Run the profiling commands first. Then create a working copy, apply the cleaning rules and validate the result. Export the clean dataset and change log to Excel. Open the output file and verify it manually. Common errors include incorrect paths, misspelled column names and missing packages.

Future Enhancements

Future versions could add a simple user interface for choosing files and cleaning rules. The script could generate a quality report with missing-value percentages and before-and-after charts. More advanced features could include date standardization, outlier review and fuzzy duplicate detection. For larger organizations, the process could be scheduled or integrated with a database. Each enhancement should be tested carefully and should preserve traceability.

Conclusion and References

This project demonstrates a simple and repeatable way to clean a small dataset using Python and Excel. It covers profiling, missing-value handling, duplicate removal, text standardization, numeric validation and export. The main lesson is that every transformation should have a reason and be documented. References: pandas documentation at https://pandas.pydata.org/docs/ ; Python documentation at https://docs.python.org/3/ ; openpyxl documentation at https://openpyxl.readthedocs.io/ . Before submission, run the code and replace illustrative counts with actual results.